Vulnerability Notes

Server-Side Request Forgery (SSRF)

Concepts, Attack Techniques, Detection, and Defenses

Reference
PortSwigger Web Security Academy

Prepared by
Youssef Reda

Contents

What Is SSRF? **1**
- Why SSRF Matters . . . 1
- SSRF vs. CSRF . . . 1

Impact of SSRF **2**
- Access to the Application Server . . . 2
- Access to Other Back-End Systems . . . 2
- Sensitive Data and Further Compromise . . . 2
- Requests to External Systems . . . 2

Common SSRF Attack Surfaces **3**
- User-Controlled URLs . . . 3
- Partial URLs and Hostname Inputs . . . 3
- URLs Embedded in Structured Data . . . 3
  - SSRF via XXE . . . 3
- The Referer Header . . . 3

SSRF Against the Server and Internal Services **4**
- SSRF Against the Server Itself . . . 4
- Exploiting Trust Relationships . . . 4
  - Internal Services and Private Addresses . . . 4
  - Administrative Interfaces and Disaster-Recovery Services . . . 4
- The Server as a Network Bridge . . . 4

Bypassing SSRF Defenses **5**
- Blacklist-Based Filters . . . 5
  - Alternative IP Representations . . . 5
  - Hostnames That Resolve to Loopback . . . 5
  - Encoding and Case Variations . . . 5
- Whitelist-Based Filters . . . 5
  - URL Parsing Inconsistencies . . . 5
  - URL Encoding and Double Encoding . . . 6
- Bypassing Filters Through Open Redirects . . . 6
- Why Filter Bypasses Work . . . 6

Blind SSRF **7**
- Overview . . . 7
- Out-of-Band Application Security Testing (OAST) . . . 7
- Burp Collaborator . . . 7
- DNS Interaction vs. HTTP Interaction . . . 7
- Limitations and Internal Probing . . . 8

Vulnerability Chaining and Further Impact 8

Testing Workflow **9**

Defensive Considerations **10**

Quick Reference **11**

What Is SSRF?

Server-Side Request Forgery (SSRF) is a vulnerability that allows an attacker to make a server-side application send a request to an unintended destination. The attacker influences the destination or content of a request that the server makes on their behalf.

Core idea:

```
Attacker -> Vulnerable application -> Target resource
```

A common example is a feature that retrieves an image or checks stock availability using a URL supplied by the user. If the destination is not properly validated, the attacker may replace the expected URL with one pointing to a local or internal service.

```
Expected:  https://stock.example/api/product/6
Modified:  http://127.0.0.1/admin
```

SSRF occurs when the application actually makes the unintended server-side request. Merely accepting or displaying a URL is not sufficient.

Why SSRF Matters

The vulnerable server may have network access, credentials, or trusted relationships that the attacker does not possess directly. Depending on the target and the application's behavior, SSRF may enable an attacker to:

- Reach internal administrative panels, APIs, and services.
- Access local endpoints on the application server.
- Retrieve sensitive data if the response is exposed.
- Interact with third-party systems from the organization's network.
- Use the server as a bridge to reach another vulnerable service.

These outcomes are possibilities, not guarantees. Impact depends on reachable destinations, the privileges of the application, and whether response content or side effects can be observed.

SSRF vs. CSRF

- **SSRF:** tricks a server-side application into sending a request to an unintended destination.
- **CSRF:** tricks a victim's browser into sending an unwanted request, potentially using the victim's authenticated session.

```
SSRF: Attacker -> Vulnerable server -> Target
CSRF: Attacker -> Victim's browser -> Target
```

Impact of SSRF

Access to the Application Server

An attacker may target services listening on the same machine as the vulnerable application. Loopback references commonly include:

```
http://127.0.0.1/
http://localhost/
```

For example, a stock-check feature might send a request to a user-provided URL:

```
POST /product/stock HTTP/1.0
Content-Type: application/x-www-form-urlencoded
stockApi=http://stock.example:8080/product/stock/check?productId=6&storeId=1
```

If validation is inadequate, the destination might be changed to:

```
stockApi=http://localhost/admin
```

If the local endpoint is reachable and its response is returned by the application, the attacker may be able to read the administrative page.

Access to Other Back-End Systems

The server may be able to reach systems on private networks that are not exposed to the Internet. Examples include internal APIs, administrative interfaces, and other web applications.

```
Attacker -> Vulnerable server -> Internal service
```

Private addresses often belong to ranges such as `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16`. Whether a target is reachable depends on the server's network configuration and outbound controls.

Sensitive Data and Further Compromise

If a reachable service returns sensitive information and the vulnerable application exposes that response, SSRF may disclose credentials, tokens, API keys, configuration, or other confidential data. SSRF may also be a step in a larger attack chain. For example, it may provide access to an internal service that contains a separate vulnerability. SSRF alone does not automatically provide Remote Code Execution (RCE).

Requests to External Systems

SSRF can cause requests to external systems to originate from the vulnerable server. This may allow unintended interactions with third-party services or make traffic appear to originate from the organization's infrastructure. The impact depends on the destination, request behavior, and applicable network restrictions.

Common SSRF Attack Surfaces

User-Controlled URLs

The most obvious attack surface is a parameter that accepts a complete URL, such as an image importer, URL previewer, webhook tester, or stock-check feature. During testing, identify where the input is used and determine whether the server actually fetches the supplied destination. Use a controlled endpoint first, and observe both the application's response and any external interaction.

Partial URLs and Hostname Inputs

An application may accept only a hostname or a URL fragment and build the final URL itself:

```
url = "https://" + host + "/api/data"
```

Even if the user cannot control the complete URL, controlling the hostname or another component may influence the server-side destination.

URLs Embedded in Structured Data

Some parsers or processing components retrieve external resources while processing structured input.

SSRF via XXE

XML External Entity (XXE) can lead to SSRF when an XML parser processes an external entity that references a URL. For example:

```
<?xml version="1.0"?>
<!DOCTYPE data [
  <!ENTITY example SYSTEM "http://internal.example/">
]>
<data>&example;</data>
```

This depends on the parser configuration and whether external entity processing is enabled. An XML endpoint is not automatically vulnerable; the relevant question is whether parsing the input triggers a server-side request.

The Referer Header

Some analytics or back-end components may retrieve a URL supplied in the `Referer` header. For example:

```
GET /product?id=10 HTTP/1.1
Host: shop.example
Referer: https://external.example/article
```

Recording the header does not itself constitute SSRF. The vulnerability exists only if a component uses the supplied value to initiate a request.

SSRF Against the Server and Internal Services

SSRF Against the Server Itself

In this pattern, the application makes a request to a service running on the same host. Common loopback identifiers include `127.0.0.1` and `localhost`. The target path may refer to an administrative endpoint or another service that trusts local requests.

```
Attacker -> Application -> Loopback interface -> Local resource
```

A successful request does not guarantee useful access: the endpoint must be listening, reachable from the relevant process, and willing to perform the requested operation.

Exploiting Trust Relationships

Some systems apply weaker authentication or access controls to requests that appear to originate from a trusted source, such as the local machine or an internal network. SSRF can abuse this trust boundary by making the request from the server's network position.

Internal Services and Private Addresses

A server-side request may target internal addresses or services on non-standard ports. Testing should establish whether the application can reach the target and whether the target's behavior provides meaningful evidence.

Administrative Interfaces and Disaster-Recovery Services

Internal administrative panels, monitoring interfaces, or disaster-recovery services may be exposed only to trusted networks. If such a service is reachable through SSRF, the security impact depends on its authentication requirements and available actions.

The Server as a Network Bridge

SSRF can bridge the gap between an external attacker and systems reachable only from the vulnerable server. The server's network access determines which targets may be reachable; the vulnerable application's response behavior determines what the attacker can learn or cause.

Bypassing SSRF Defenses

Input filters often attempt to block dangerous destinations. Bypasses become possible when validation checks a textual representation of the input rather than the normalized destination that the HTTP client ultimately contacts.

Blacklist-Based Filters

A blacklist may reject strings such as:

```
127.0.0.1
localhost
/admin
```

This approach is fragile because the same destination may be represented in different ways, and different components may parse or normalize the input differently.

Alternative IP Representations

Some URL parsers and networking libraries accept alternative representations of the IPv4 loopback address, including:

```
127.0.0.1
2130706433
017700000001
127.1
```

Acceptance depends on the parser and networking implementation. These forms should be treated as parser-dependent test cases, not universal bypasses.

Hostnames That Resolve to Loopback

A filter that checks only the literal strings `127.0.0.1` and `localhost` may miss a hostname that resolves to a loopback address. For example, PortSwigger discusses `spoofed.burpcollaborator.net` in this context. Checking a hostname as a string is not equivalent to validating the IP address to which it resolves. DNS behavior and the time of resolution can affect the outcome.

Encoding and Case Variations

URL encoding can represent characters using percent-encoded sequences. For example, `%2F` represents a forward slash (`/`). If one component validates encoded input and another decodes it later, the two stages may interpret the same input differently. Case variation may also expose weak, case-sensitive string checks. However, case changes are not a universal bypass: effectiveness depends on the value being checked and how the application interprets it.

Whitelist-Based Filters

A whitelist permits only destinations that match predefined rules, such as an approved hostname. It is generally more restrictive than a blacklist, but it can still fail when implemented with simple string matching or inconsistent URL parsing.

URL Parsing Inconsistencies

A URL should be parsed using a well-defined parser, and validation should apply to the parsed components rather than a substring of the raw input. Pay particular attention to the hostname, port, scheme, user-info component, path, and the final destination selected by the HTTP client.

The key question is whether the validator and the request-making component interpret the URL in the same way. Parser differences can cause a filter to approve one apparent destination while the client connects to another.

URL Encoding and Double Encoding

Encoding can create inconsistencies when different components decode input a different number of times or at different stages. Testing should determine how the application normalizes the input before validation and before making the request. Double encoding is not inherently a bypass; it matters only when the processing chain handles the representations inconsistently.

Bypassing Filters Through Open Redirects

An open redirect allows an attacker to influence a redirect destination. If an SSRF filter validates only the initial URL and the server-side HTTP client follows redirects, the request may reach a destination that would otherwise be blocked.

```
Allowed URL -> Redirect response -> Restricted destination
```

A possible sequence is:

1. The application validates the initial URL.
2. The initial host is accepted.
3. The server makes the request.
4. The server receives a redirect response.
5. The HTTP client follows the redirect without adequate revalidation.

Common redirect status codes include `301`, `302`, `303`, `307`, and `308`. Client behavior varies, including how methods and request bodies are handled. A redirect-based bypass requires a usable redirect path and a client that follows it.

Why Filter Bypasses Work

A request may pass through several stages:

```
User-supplied URL
       |
       v
Input validation
       |
       v
URL parsing / normalization
       |
       v
DNS resolution
       |
       v
Redirect handling
       |
       v
Final connection destination
```

If validation does not account for these stages, the filter may approve an input that eventually reaches a restricted destination. The underlying issue is a mismatch between the representation checked by the filter and the destination actually contacted.

Blind SSRF

Overview

Blind SSRF occurs when an attacker can trigger a server-side request but cannot directly see the response from the target in the application's response. The request may still have effects, but the lack of visible response content limits what can be learned.

- **Normal SSRF:** the application exposes some of the target response, such as its body or status.
- **Blind SSRF:** the request occurs, but the target response is not directly returned to the attacker.

A generic message such as `Request sent successfully` does not show whether the target returned useful content.

Out-of-Band Application Security Testing (OAST)

OAST detects interactions that happen outside the application's normal response. During an authorized test, supply a unique URL pointing to an endpoint you control or a tool-provided interaction domain, then check whether the request-processing chain contacts it.

```
Tester -> Vulnerable application -> Monitored endpoint
                                          |
                                          v
                                  Interaction recorded
```

An observed interaction is evidence that a component contacted or resolved the supplied domain. It does not, by itself, prove that an internal resource is exploitable.

Burp Collaborator

Burp Collaborator is an OAST tool that provides unique domains and records interactions associated with them. A basic testing workflow is:

1. Generate a unique Collaborator domain.
2. Place it in a suspected SSRF input.
3. Submit the request to the application.
4. Check Collaborator for incoming interactions.
5. Interpret the interaction type and timing in context.

Example input:

```
stockApi=http://<unique-id>.burpcollaborator.net/
```

Use a unique identifier for each test where practical, so that observed interactions can be correlated with the request that triggered them.

DNS Interaction vs. HTTP Interaction

DNS resolution usually occurs before an HTTP connection can be made:

```
Hostname -> DNS lookup -> IP address -> HTTP connection
```

Possible observations include:

- **DNS and HTTP interaction:** the domain was resolved and an HTTP request reached the monitored endpoint.
- **DNS only:** a lookup occurred, but no HTTP interaction was observed.

DNS-only interactions may result from outbound HTTP filtering, connection failures, protocol restrictions, or the application's behavior. A DNS lookup alone does not prove that an HTTP request reached the destination. Conversely, no recorded interaction does not conclusively prove that no request was attempted.

Limitations and Internal Probing

Without a visible response, an attacker cannot directly read internal pages, inspect target response headers, or reliably distinguish successful and failed requests unless another signal is available. Blind SSRF may still help probe internal systems if there is an observable side channel, such as an out-of-band interaction or another measurable difference. If all responses and effects are hidden, reliable enumeration may not be possible.

Vulnerability Chaining and Further Impact

Blind SSRF may provide a way to reach a target that the attacker cannot access directly. If that target contains a separate vulnerability, the two issues may be chained:

```
Blind SSRF -> Reach internal service
           -> Exploit separate vulnerability
           -> Potential further impact
```

Blind SSRF does not automatically lead to RCE. A second weakness and the conditions required to exploit it must exist. An attacker-controlled server may also return a crafted HTTP response that exercises a vulnerability in the application's HTTP client or response-processing logic. The outcome depends on the client implementation and the presence of a separate exploitable weakness; a crafted response does not inherently produce code execution.

Testing Workflow

Use this checklist when investigating a suspected SSRF issue in an authorized environment:

1. **Identify inputs:** look for full URLs, hostnames, partial URLs, URL imports, webhook features, XML input, and relevant headers.
2. **Understand the request path:** determine which component consumes the input and whether it makes a server-side request.
3. **Start with a controlled destination:** use a domain or endpoint you control to test for outbound interaction.
4. **Compare observable behavior:** review the response, timing, status changes, and out-of-band interactions.
5. **Test validation boundaries:** in a lab or authorized scope, assess how the application handles loopback references, private addresses, URL parsing, encoding, DNS resolution, and redirects.
6. **Assess impact carefully:** distinguish confirmed request behavior from assumptions about data disclosure, internal access, or further exploitation.
7. **Record evidence:** save the relevant request, response, payload, and observed interaction so the finding can be reproduced.

Defensive Considerations

The weaknesses described above show why checking a few blocked strings is not sufficient. Effective SSRF prevention should validate the destination that will actually be contacted, not merely the original input text.

- Prefer an allowlist of explicitly approved destinations when the feature requires only a known set of hosts.
- Parse and normalize URLs consistently before validating their components.
- Resolve hostnames and validate the resulting addresses against the intended network policy.
- Revalidate each redirect destination, or disable automatic redirects when they are not required.
- Apply outbound network restrictions so the application cannot reach unnecessary internal services.
- Avoid exposing raw responses from fetched resources when the feature does not need to return them.
- Treat DNS results and URL parser behavior as part of the security boundary, not as string-matching details.

These controls should be designed together. Application-level validation and network-level restrictions provide complementary protection.

Quick Reference

Topic	Remember
SSRF	The server makes a request to a destination influenced by the attacker.
Impact	Depends on reachable systems, application privileges, and observable behavior.
Blacklists	Can fail because of alternate representations and inconsistent normalization.
Whitelists	Must validate parsed URL components and the actual destination.
Redirects	Revalidate the destination after every redirect or disable redirects.
Blind SSRF	Use OAST to detect interactions when the target response is hidden.
DNS-only result	Confirms a lookup, not necessarily a successful HTTP request.
RCE	Requires a separate exploitable weakness; SSRF alone does not guarantee RCE.